

Nome Progetto: MCPwn

1. Composizione del Gruppo:

Camonita Dario

2. Linguaggi di programmazione:

- Go
- Python
- C++

3. Descrizione della Progettazione e dell'Implementazione:

3.1 Obiettivo dell'applicazione:

MCPwn è un'applicazione backend concepita per l'esecuzione remota di strumenti di sicurezza informatica facilmente utilizzabili via CLI, come nmap, gobuster, nikto, etc...

L'obiettivo principale è fornire una piattaforma centralizzata e modulare per orchestrare tool specifici, facilitandone l'integrazione con script di automazione e GUI. Sostanzialmente, il sistema agisce come un livello di astrazione, permettendo di lanciare strumenti a riga di comando attraverso una semplice chiamata API.

Questo Proof of Concept implementerà *almeno* tutte le funzionalità base ma **potrebbe non essere sicuro per l'uso in ambienti di produzione**, in quanto potrebbe contenere delle potenziali esecuzioni di codice remoto (RCE), dato che per ogni tool andrebbero implementate delle specifiche funzioni di validazione e sanitizzazione dei parametri in ingresso (allow list).

3.2 Funzioni da implementare:

Le funzionalità principali del sistema saranno:

- **Esposizione di un'API RESTful:** fornire endpoint HTTP per ricevere richieste di esecuzione comandi. Tramite cURL o strumenti simili, quindi, gli utenti potranno inviare richieste per eseguire specifici tool di sicurezza.
- **Gestione e instradamento delle richieste:** ricevere il traffico in ingresso e smistarlo correttamente verso il componente logico interno.
- **Orchestrazione dei comandi:** validare dove necessario le richieste in ingresso e coordinare l'esecuzione degli strumenti richiesti.
- **Esecuzione isolata dei processi:** per avviare i comandi in processi separati, gestendo in modo affidabile la cattura dell'output (`stdout/stderr`) e applicando limiti di tempo (timeout) per prevenire stalli.
- **Interfacciamento tra moduli:** garantire una comunicazione stabile ed efficiente tra i diversi componenti dell'architettura.

3.3 Architettura dell'applicazione e linguaggi utilizzati:

L'applicazione adotta un'**architettura a microservizi**. Questo modello prevede la scomposizione dell'applicazione in servizi indipendenti e specializzati, ciascuno sviluppato con il linguaggio di programmazione più adatto al suo compito specifico. Sarà creato anche un **Makefile** (e potenzialmente dei **Dockerfile** gestiti con **docker-compose**) per facilitare la build e l'esecuzione dei vari moduli.

Modulo 1: API Gateway

- **Linguaggio: Go**
- **Funzioni:** questo modulo si pone come entry point unico per tutte le richieste esterne usando principalmente la libreria **net/http**. Il suo ruolo sarà:
 - Gestire in modo efficiente e concorrente un elevato volume di traffico HTTP.
 - Fungere da reverse proxy, instradando le chiamate API in arrivo verso l'API Server (Modulo 2).
 - Fornire un endpoint stabile e centralizzato per i client del sistema.

Modulo 2: API Server

- **Linguaggio: Python**
- **Funzioni:** rappresenta la logica dell'applicazione, con il compito di *orchestrare* le operazioni. Potenzialmente verranno adottate librerie esterne come **Pydantic** e **FastAPI** in quanto le sue funzioni saranno:
 - Esporre gli endpoint dell'API RESTful per l'interazione con i vari tool.
 - Validare i dati e i parametri ricevuti nelle richieste.
 - Interfacciarsi con il motore di esecuzione (Modulo 3) per delegare l'effettivo avvio dei comandi.

Modulo 3: Esecuzione di tool

- **Linguaggio: C++**
- **Funzioni:** questo modulo è il core del progetto in quanto si occuperà di eseguire i comandi, già presenti nel sistema; per interfacciarsi con gli altri moduli, verrà buildato come libreria (**.dylib** o **.so**, dipende dal sistema operativo). Le sue responsabilità principali saranno:
 - Eseguire comandi in processi isolati per garantire la stabilità del sistema.
 - Applicare timeout per evitare che processi anomali blocchino il sistema.
 - Catturare lo stdout e stderr dei processi eseguiti.
 - Esporre un'interfaccia stabile (secondo l'ABI del C) per permettere all'API Server, descritto in precedenza, di comunicare con questa libreria nativa.

3.4 API esterne:

Questo progetto è intenzionalmente creato per **non dipendere da alcuna API esterna**. Il suo valore consiste nel fornire un'**API interna** che possa essere utilizzata come un componente utile al fine di costruire applicazioni più complesse e complete, magari come sistemi di automazione per penetration testing o client che fanno uso di LLM/IA.